

RAG

Class 1: Foundation

What is RAG? · Why is it Needed? · How Does it Work?

WHAT YOU WILL LEARN IN THIS CLASS:

- What is RAG — Clear definition with 3 real-world analogies
- Why RAG is Needed — 5 LLM problems explained with examples
- How RAG Works — Complete pipeline walkthrough (Indexing + Query)
- Who Uses RAG — Perplexity AI, ChatGPT, Copilot, NotionAI and more
- RAG vs Traditional Search vs Vanilla LLM — Clear comparison
- Key Terminology — Embeddings, Vectors, Chunks, Similarity Search

C1
Foundation

C2
Indexing

C3
Retrieval

C4
Generation

C5
Advanced

C6
Production

6-Class RAG Mastery Roadmap — You Are Here: Class 1

Class 1 — Table of Contents

- 1.1 What is RAG?** — *Clear definition with 3 analogies to remember forever*
- 1.2 Why RAG is Needed** — *5 critical LLM failures explained with real examples*
- 1.3 How RAG Works** — *Complete 2-phase pipeline explained step by step*
- 1.4 The Indexing Phase** — *How documents become searchable (conceptual overview)*
- 1.5 The Query Phase** — *How questions get answered (conceptual overview)*
- 1.6 Key RAG Terminology** — *Every important term defined with simple language*
- 1.7 Who Uses RAG in 2025-26?** — *Real products & companies using RAG right now*
- 1.8 RAG vs Others** — *RAG vs Fine-Tuning vs Traditional Search vs Vanilla LLM*
- 1.9 Summary & What's Next** — *Key takeaways and preview of Class 2*

1.1 | What is RAG?

Clear definition with 3 analogies you'll never forget

The Full Form

RAG stands for **Retrieval-Augmented Generation**. Let's break down each word:

Word	Meaning	In Simple Terms
Retrieval	The process of searching and finding relevant information from a knowledge base	Like using Google to search for something before answering
Augmented	Enhanced or improved by adding something extra	The LLM gets 'boosted' with extra knowledge it didn't have
Generation	The LLM producing a natural language answer	GPT/Claude/Llama writes a human-like response

The Official Definition

Definition

RAG is an AI architecture that improves LLM answers by first **searching a knowledge base** to find relevant documents, then **feeding those documents** to the LLM along with the user's question, so the LLM generates its answer **based on actual reference material** rather than its training memory alone.

It was first proposed in a 2020 research paper by Meta AI titled 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks'.

The Formula (Simple)

```
// The RAG Formula
```

```
RAG Answer = LLM( Your Question + Retrieved Documents )
```

```
Without RAG: LLM answers from MEMORY only (may be wrong)
```

```
With RAG: LLM answers from DOCUMENTS + MEMORY (accurate, cited)
```

Analogy 1: The Open-Book Exam

Imagine two students taking the same exam:

	Student A (Vanilla LLM)	Student B (RAG)
Exam Type	Closed-book exam — must answer everything from memory	Open-book exam — can look up the textbook before answering
Preparation	Studied months ago, might have forgotten details	Has the textbook right there, can find the exact page

	Student A (Vanilla LLM)	Student B (RAG)
Answer Quality	Gets the general idea right but may confuse specific details, dates, or numbers	Cites exact facts, correct numbers, references specific sections
Confidence	Sounds confident even when wrong (this is hallucination!)	Can point to exactly where the answer came from
When Info Changes	Still gives the old answer (outdated knowledge)	Looks up the latest edition of the textbook (always current)

RAG gives the LLM 'open-book exam' access to your documents.

Analogy 2: The Research Assistant

Think of RAG like a brilliant research assistant who helps a CEO answer questions:

Without RAG (CEO answers alone)

CEO gets a question: 'What were our Q3 sales numbers in the APAC region?'

The CEO tries to remember from a meeting 3 months ago... 'I think it was around \$12 million... or was it \$14 million?'

Result: Plausible-sounding but potentially wrong answer. No source to verify.

With RAG (CEO + Research Assistant)

CEO gets the same question. Research assistant quickly searches the company database, finds the Q3 sales report, pulls up the APAC section, and hands it to the CEO.

CEO reads the report and says: 'Q3 APAC sales were \$13.2 million, up 8% from Q2. Source: Q3 Sales Report, Page 14.'

Result: Accurate, specific, cited answer. The research assistant is the 'Retrieval' part of RAG.

Analogy 3: Google Search + ChatGPT Combined

This is the simplest way to explain RAG to anyone:

Approach	What Happens	Result
Google Search Only	You search, get 10 blue links. You must read all pages yourself and figure out the answer.	Accurate sources but YOU do all the work. Time-consuming.
ChatGPT Only	You ask a question, ChatGPT answers from memory. No searching. No sources cited.	Easy and fast, but may be wrong or outdated. No proof.

Approach	What Happens	Result
RAG (Search + AI)	System searches your documents first (like Google), finds the relevant pages, then AI reads them and gives you a clean summary with citations.	Best of both worlds: accurate, sourced, AND easy to understand.

Real Example — Perplexity AI

Perplexity AI is the most famous RAG product. When you ask it a question:

1. It searches the internet (Retrieval)
2. It reads the top results (Augmentation)
3. It generates a clear answer with numbered citations (Generation)

That's exactly RAG — applied to the entire internet as the knowledge base!

1.2 | Why RAG is Needed

5 critical LLM failures — each one explained in detail

LLMs like GPT-5, Claude 4.6, Gemini 3, and Llama 4 are incredibly powerful. They can write code, explain complex topics, and even pass medical and legal exams. But they have **5 critical flaws** that make them unreliable for production use without RAG. Let's understand each one deeply:

Problem 1: Hallucination (Making Up Facts)

What is hallucination? When an LLM generates information that sounds perfectly correct and confident but is completely made up. The LLM doesn't 'know' it's lying — it's just predicting the most likely next token based on patterns in its training data.

Real Example — Hallucination in Action

User asks ChatGPT: 'What is the refund policy for Acme Corp?'

ChatGPT responds: 'Acme Corp offers a 30-day money-back guarantee on all products. You can initiate a return through their website at acme.com/returns.'

Reality: Acme Corp actually has a 14-day exchange-only policy with no money-back option. The URL acme.com/returns doesn't exist.

The LLM invented a plausible-sounding answer because it was never trained on Acme Corp's actual policy. A customer following this advice would be completely wrong.

How RAG Fixes This

With RAG: The system first searches Acme Corp's actual policy document, retrieves the relevant section, and the LLM generates: 'Acme Corp offers a 14-day exchange-only policy. No cash refunds. (Source: [Acme_Return_Policy.pdf](#), Page 3)'

The LLM can ONLY use what was retrieved — it can't invent facts because the system prompt says: 'Answer ONLY from the provided context.'

Problem 2: Knowledge Cutoff (Outdated Information)

What is knowledge cutoff? Every LLM is trained on data up to a certain date. After that date, the model has zero knowledge of new events, updated policies, new products, or changed regulations.

LLM	Knowledge Cutoff	What It Doesn't Know
GPT-5.5 (latest, Apr 2026)	December 2025	Events after Dec 2025 — still a 5-month gap as of May 2026
GPT-4o (still widely used)	October 2023	Misses all of 2024 and 2025 — a 30-month gap!

LLM	Knowledge Cutoff	What It Doesn't Know
Claude 4.6 Opus (latest)	August 2025	Events after Aug 2025 not in training data
Gemini 3.1 Flash	January 2025	Misses most of 2025 entirely
Llama 4 (Meta, open source)	August 2024	Nearly 2 years behind as of May 2026

How RAG Fixes This

With RAG, your knowledge base is a separate database that you control. You can:

- Add new documents any time (daily, hourly, or in real-time)
- Update existing documents when policies change
- Remove outdated documents

The LLM always gets the LATEST information because it reads from your updated knowledge base, not from its frozen training data.

Problem 3: No Access to Private/Internal Data

The core issue: LLMs are trained on public internet data. They have never seen your company's internal documents — HR policies, product specs, customer databases, financial reports, internal wikis, Slack conversations, or proprietary research. So they simply cannot answer questions about your private data.

Real-World Scenario

An employee at TechCorp asks: 'What is our work-from-home policy?'

Without RAG: GPT-4 will either refuse ('I don't have access to TechCorp policies') or worse, hallucinate a generic WFH policy that doesn't match TechCorp's actual rules.

With RAG: The system searches TechCorp's indexed HR documents, finds Section 5.2 of the Employee Handbook, and generates: 'TechCorp allows 3 WFH days per week (Mon/Wed/Fri). Manager approval required for additional days. (Source: Employee Handbook 2024, Section 5.2)'

Problem 4: No Source Citations (Can't Prove Its Answer)

Why this matters: In many industries — legal, medical, finance, compliance, education — an answer without a source is worthless. You need to know WHERE the information came from so you can verify it, audit it, and take responsibility for acting on it.

Industry	Why Citations Matter	Risk Without Citations
Legal	Lawyers need case references, statute numbers, exact clause text	Citing a non-existent law in court = malpractice
Medical	Doctors need drug interaction evidence, clinical trial references	Wrong dosage from an uncited source = patient harm

Industry	Why Citations Matter	Risk Without Citations
Finance	Auditors need regulation references, compliance document citations	Failing compliance audit = fines, legal action
HR	Employees need policy section, page number, effective date	Wrong policy guidance = grievance, lawsuit

How RAG Provides Citations

Every chunk in RAG carries **metadata** — the source file name, page number, section heading, and document date. When the LLM generates an answer, it includes these citations:

'Your annual leave is 24 days. (Source: HR_Policy_2024.pdf, Section 3.2, Page 12)'

The user can click through and verify the exact source. This is impossible with vanilla LLMs.

Problem 5: Context Window Limits

What is a context window? It's the maximum amount of text an LLM can process in a single request. Think of it like the LLM's 'working memory'. Even the largest models have limits:

Model	Context Window	Approximate Page Equivalent
GPT-5.5	256K tokens	~640 pages
Claude 4.6 Opus	200K tokens	~500 pages
Gemini 3.1 Pro	2M tokens	~5000 pages (largest in 2026!)
GPT-4o (still popular)	128K tokens	~300 pages
Llama 4 (open source)	128K tokens	~300 pages

Even 300 pages might seem like a lot, but enterprise documents can be **thousands of pages**. A legal firm might have 50,000 case files. A hospital might have millions of patient records. You can't fit all of that into one prompt.

How RAG Fixes This

RAG never sends the entire knowledge base to the LLM. Instead:

1. It finds **ONLY** the 3-5 most relevant chunks (maybe 2000 characters total)
2. Sends just those small chunks + the question to the LLM
3. The LLM only processes what it needs — never exceeding context limits

This means RAG can work with knowledge bases of any size — 100 pages or 10 million pages.

1.3 | How RAG Works

The complete 2-phase pipeline — conceptual walkthrough

The Big Picture

Every RAG system in the world — whether it's Perplexity AI, a startup's chatbot, or a Fortune 500 enterprise assistant — follows the **exact same two-phase architecture**:

Phase	When It Runs	Purpose	Analogy
Phase 1: Indexing	Once (or when documents change)	Convert all your documents into a searchable format	Like organizing a library — shelving books, creating a catalog
Phase 2: Querying	Every time a user asks a question	Search the knowledge base and generate an answer	Like a librarian who finds the book, reads the page, and gives you the answer

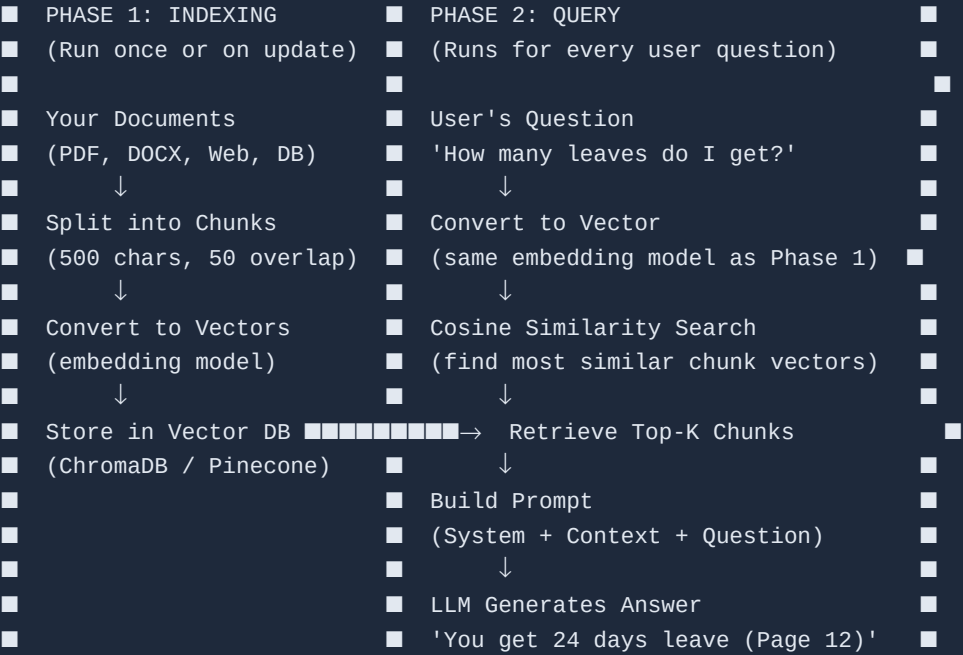
The Complete Pipeline — Step by Step

INDEXING: Documents → Chunk / Split → Embed (→ Vectors) → Store in Vector DB

QUERYING: User Question → Embed Query → Search Vector DB → Top-K Chunks → LLM → Answer

Full Architecture Diagram

// Complete RAG Architecture – Phase 1 connects to Phase 2 via the shared Vector Database



1.4 | The Indexing Phase

How documents become searchable — conceptual overview

Indexing is the 'preparation' step. Before anyone asks a question, we need to process all our documents into a searchable format. Think of it like setting up a library before it opens to the public.

Step 1: Loading Documents

First, we read all the raw documents. RAG systems can handle almost any format — PDFs, Word documents, web pages, CSVs, databases, Notion pages, emails, and more. Each document gets converted into plain text along with **metadata** (which file it came from, which page, what date it was created, etc.).

Example

Input: hr_policy_2024.pdf (200 pages)

Output: 200 text blocks, each labeled with:

- Source: hr_policy_2024.pdf
- Page: 1, 2, 3... 200
- Loaded at: 2024-11-15

Step 2: Chunking (Splitting into Smaller Pieces)

A single page can contain 3000+ characters covering multiple topics. If we search against entire pages, we'll get results that are mostly irrelevant with a small relevant section buried inside. So we split each page into **chunks** — smaller pieces of 300-600 characters each.

Why Chunking Matters — A Visual Example

Imagine Page 12 of the HR policy contains:

- Paragraph about annual leave (what we want)
- Paragraph about sick leave
- Paragraph about maternity leave
- A table of public holidays

Without chunking: The entire page is one result. The LLM gets all 4 topics and may get confused.

With chunking: Each paragraph becomes a separate chunk. When someone asks about annual leave, **ONLY** the annual leave chunk is retrieved — clean, precise, no noise.

Chunk overlap: When we split text, a sentence might get cut in half at the boundary. To prevent this, we add an **overlap** — each chunk shares 50 characters with the next chunk. This ensures no information is lost at the edges.

Step 3: Embedding (Converting Text to Numbers)

Computers can't directly compare text for 'meaning'. The word 'vacation' and 'annual leave' look completely different as text, but mean the same thing. **Embedding** solves this by converting text into a list of numbers (called a **vector**) that captures its meaning. An embedding model is a neural network trained

on billions of text pairs to learn what 'similar meaning' looks like.

How Embeddings Capture Meaning

Text	Vector (simplified)	Similar To Query?
Query: 'vacation days per year'	[0.78, 0.82, 0.15, 0.91, ...] → 1536 numbers	— (this IS the query)
'Annual leave entitlement is 24 days'	[0.76, 0.80, 0.17, 0.89, ...] → VERY close!	YES — same meaning, different words!
'Sick leave requires doctor certificate'	[0.52, 0.60, 0.44, 0.71, ...]	SOMEWHAT — related topic (leave)
'Python is a programming language'	[0.12, 0.31, 0.89, 0.24, ...]	NO — completely different topic

This is the magic of embeddings — 'vacation days' and 'annual leave entitlement' have similar vectors even though they share zero words.

Step 4: Storing in a Vector Database

All the vectors (along with the original text and metadata) are stored in a **vector database**. This is a special database optimized for one thing: given a query vector, find the most similar stored vectors as fast as possible. Regular databases search by exact match (WHERE name = 'John'). Vector databases search by **similarity** (find vectors closest to this one).

Think of It Like This

A regular database is like searching a phone book by name — exact match only.

A vector database is like asking 'find me people who LIVE NEAR this address' — it finds the closest matches even if they're not an exact match.

Popular vector databases: ChromaDB (easy, free), Pinecone (managed cloud), FAISS (Meta's library, very fast), Qdrant, Weaviate, Milvus.

1.5 | The Query Phase

How questions get answered — step by step walkthrough

This phase runs every time a user asks a question. Let's walk through a complete example:

Scenario

Knowledge base: AmanCorp's HR Policy (200 pages, indexed in Phase 1)

User's question: 'How many annual leaves do I get and can I carry them forward?'

Step	What Happens	In Our Example
Step 1: Receive Query	The user types a natural language question into the chatbot interface.	User types: 'How many annual leaves do I get and can I carry them forward?'
Step 2: Embed the Query	The SAME embedding model from indexing converts the question into a vector. This is critical — you must use the same model for both phases.	Question → [0.78, 0.82, 0.15, ...] (1536 numbers)
Step 3: Similarity Search	The vector database compares the query vector against ALL stored chunk vectors using cosine similarity. It finds the chunks whose vectors are most similar.	Chunk about 'annual leave = 24 days' → similarity: 0.94 (TOP MATCH) Chunk about 'carry forward up to 90 days' → similarity: 0.89 (MATCH) Chunk about 'sick leave' → similarity: 0.45 (not relevant, discarded)
Step 4: Retrieve Top-K	The top K most similar chunks are selected (typically K=3 to 6). These chunks, along with their metadata, become the 'context' for the LLM.	Retrieved: • Chunk 1: 'All employees get 24 days annual leave...' (Page 12) • Chunk 2: 'Unused leave carried forward for 90 days...' (Page 13)
Step 5: Build Prompt	A prompt is assembled with 3 parts: system instructions ('answer only from context'), the retrieved chunks as context, and the user's question.	System: 'Answer only from context. Cite page numbers.' Context: [Chunk 1] + [Chunk 2] Question: 'How many annual leaves...'
Step 6: LLM Generates	The LLM reads the prompt and generates an answer BASED ON the retrieved context. It cites the source documents.	Answer: 'You are entitled to 24 days of annual leave (Page 12). Unused leave can be carried forward for up to 90 days into the next year (Page 13).'

Notice how the LLM only used information from the retrieved chunks — it didn't make anything up.

1.6 | Key RAG Terminology

Every important term you must know — defined simply

Term	Simple Definition	Analogy
RAG	Retrieval-Augmented Generation — search first, then generate answer using found docs	Open-book exam for AI
Embedding	A list of numbers (vector) that represents the meaning of text	Like a GPS coordinate for meaning — similar meanings are close together
Vector	A list of floating-point numbers. For text, typically 768-1536 numbers per chunk.	A point in high-dimensional space
Chunk	A small piece of text (300-600 chars) split from a larger document	One paragraph or section from a book
Chunk Overlap	Characters shared between consecutive chunks to prevent context loss at boundaries	Like overlapping tiles on a roof — no gaps
Vector Database	A database optimized for storing and searching vectors by similarity	A smart library catalog that finds 'similar meaning' books, not just exact titles
Cosine Similarity	A math formula that measures how similar two vectors are. Range: 0 (different) to 1 (identical).	Like measuring the angle between two arrows — same direction = similar
Top-K	The K most relevant chunks returned from similarity search (usually K=3 to 6)	The top 5 search results
Context Window	The maximum amount of text an LLM can process in one request	The LLM's short-term memory capacity
Hallucination	When an LLM confidently generates false information	A student making up facts on an exam instead of saying 'I don't know'
Knowledge Cutoff	The date after which an LLM has no training data	The last day the textbook was updated
Indexing	The process of converting documents into searchable vectors (Phase 1 of RAG)	Organizing and cataloging a library
Retrieval	The process of finding relevant chunks for a given query (Phase 2, Step 3)	A librarian finding the right book for your question
Augmentation	Adding retrieved context to the LLM prompt before generation	Giving the student reference notes before they answer

Term	Simple Definition	Analogy
Generation	The LLM producing a final answer based on the augmented prompt	The student writing their answer using the reference notes

1.7 | Who Uses RAG in 2025-26?

Real products and companies using RAG right now (May 2026)

RAG is not just a research concept — it powers some of the most popular AI products in the world. Here are real examples across different categories:

Product / Company	Category	How They Use RAG
Perplexity AI	AI Search Engine	The most famous RAG product. Searches the internet, retrieves relevant web pages, generates cited answers. Now processes 100M+ queries/month in 2026.
ChatGPT (GPT-5 + Browse)	General AI Assistant	GPT-5's deep research mode uses RAG — it searches the web, reads multiple sources, then generates a comprehensive cited answer.
GitHub Copilot Chat	Code Assistant	Uses RAG over your entire codebase. Retrieves relevant code, functions, and docs from your repo before generating suggestions.
Google AI Overviews / Gemini	Search + AI	Google's AI Overviews (powered by Gemini 3.1) use RAG — retrieve web results first, then generate AI summaries at the top of search.
Notion AI (2025+)	Workspace AI	Indexes all Notion pages and databases. Uses RAG to answer questions from YOUR workspace data — not public internet.
Amazon Q Business	Enterprise AI	AWS's enterprise RAG platform. Indexes S3, Confluence, SharePoint, Slack. Used by thousands of enterprises in 2025-26.
Harvey AI	Legal AI	Raised \$300M+ by 2025. Uses RAG over case law, contracts, and firm documents. Lawyers get cited answers from their own data.
Microsoft 365 Copilot	Office AI	RAG over your emails, Teams, OneDrive, SharePoint. Microsoft Graph index powers retrieval before Copilot generates responses.
Glean	Enterprise Search + AI	Valued at \$4.6B in 2025. Indexes 100+ enterprise apps (Slack, Jira, Drive). AI answers from your company's internal knowledge.
OpenAI Assistants API	Developer Platform	File search tool built into the API — developers build custom RAG apps with built-in vector store. Powers thousands of custom chatbots.

Key Takeaway

Every AI product that 'searches before answering' is using some form of RAG. It's the single most widely adopted technique for making LLMs useful in the real world. If you understand RAG deeply, you understand how 90% of production AI assistants work.

1.8 | RAG vs Others

RAG vs Fine-Tuning vs Traditional Search vs Vanilla LLM

The 4-Way Comparison

Aspect	Vanilla LLM	Traditional Search	Fine-Tuned LLM	RAG
How it works	Answers from training memory only	Keyword search returns document links	Retrain the LLM on your domain data	Search docs first, then LLM answers from them
Knowledge source	Frozen at training date	Your indexed documents	Baked into model weights	Your indexed documents (always updatable)
Can update knowledge?	No — requires full retraining	Yes — re-index documents	Partial — expensive retraining needed	Yes — just add/update docs in vector DB
Hallucination risk	HIGH — makes up facts confidently	None — shows raw documents only	MEDIUM — reduced but still possible	LOW — grounded in retrieved documents
Source citations	Not possible	Yes — shows document links	Not possible	Yes — chunk metadata (file, page)
Private data access	No	Yes — indexes your files	Yes — but frozen at training time	Yes — always up-to-date
Answer quality	Often wrong for specific/niche queries	No answer — just links (user reads themselves)	Good for trained domain, may fail on new topics	High — accurate, cited, specific
Setup cost	Near zero (API call)	Low to medium	Very high (GPU training: \$100-\$10K+)	Low to medium (embedding + vector DB)
Setup time	Minutes	Hours to days	Days to weeks	Hours to days
Best for	General conversation, brainstorming	Document search without AI summary	Custom style, format, or behavior	Accurate Q&A; over documents with citations

When to Use What?

Decision Framework

Use VANILLA LLM when: You need general conversation, brainstorming, creative writing — and accuracy of specific facts doesn't matter.

Use TRADITIONAL SEARCH when: You want users to find and read documents themselves — no AI-generated answers needed.

Use FINE-TUNING when: You need the LLM to adopt a specific style, format, or behavior (e.g., always respond like a doctor, always output JSON, specific brand voice).

Use RAG when: You need accurate, cited answers over private/changing data. This covers 80% of enterprise AI use cases.

Use RAG + FINE-TUNING when: You need domain-specific behavior AND real-time knowledge access. This is the gold standard for production enterprise AI.

1.9 | Summary & What's Next

Key takeaways from Class 1 + preview of Class 2

Class 1 — Key Takeaways

- **RAG = Retrieve + Augment + Generate.** Search your docs first, feed them to the LLM, then generate a cited answer.
- **RAG solves 5 LLM problems:** hallucination, knowledge cutoff, no private data, no citations, context limits.
- **Two phases:** Indexing (documents → chunks → vectors → vector DB) and Querying (question → search → top-K → LLM → answer).
- **Embeddings are the magic:** They convert text into numbers that capture meaning, enabling semantic search beyond keywords.
- **Vector databases** are specialized stores for similarity search across millions of vectors in milliseconds.
- **RAG is everywhere:** Perplexity AI, ChatGPT Browse, Copilot, Google AI Overviews — all use RAG.
- **RAG beats vanilla LLMs** for any task requiring accuracy, citations, private data, or up-to-date information.
- **RAG + Fine-Tuning combined** is the gold standard for enterprise AI deployments.

Quick Revision — The RAG Pipeline in 6 Lines

// RAG Pipeline – Memorize This

1. **LOAD:** Read your documents (PDF, DOCX, web pages, databases)
2. **CHUNK:** Split into small overlapping pieces (300-600 chars)
3. **EMBED:** Convert each chunk to a vector (1536 numbers)
4. **STORE:** Save vectors in a vector database (ChromaDB, Pinecone)
5. **SEARCH:** When user asks, embed query + find most similar chunks
6. **GENERATE:** LLM reads retrieved chunks + generates cited answer

Preview of Class 2: Indexing Pipeline Deep Dive

What We'll Cover in Class 2

In Class 2, we'll go hands-on with the Indexing Pipeline:

- Document Loaders — loading PDFs, web pages, databases, CSVs, Notion, Confluence
- Chunking Deep Dive — all 7 strategies compared with code examples
- Embeddings Deep Dive — 2026 models (Qwen3-Embedding, Voyage AI, Cohere Embed v4, BGE-M3)
- Vector Databases Deep Dive — setting up ChromaDB, understanding HNSW algorithm
- Hands-on — indexing a real 100-page PDF step by step

Make sure you've understood the concepts from Class 1 before moving to Class 2!

6-Class Roadmap

Class	Title	Focus
Class 1 (TODAY)	Foundation — What, Why & How	RAG concept, pipeline overview, terminology, real-world examples
Class 2	Indexing Pipeline	Document loading, chunking strategies, embeddings, vector databases
Class 3	Retrieval Techniques	Dense, sparse, hybrid retrieval, reranking, MMR, metadata filtering
Class 4	Generation & Prompt Design	Prompt anatomy, chain types, citation strategies, full end-to-end build
Class 5	Advanced RAG Patterns	HyDE, Self-RAG, CRAG, GraphRAG, Agentic RAG, multi-modal RAG
Class 6	Production & Interview Prep	RAGAS evaluation, production stack, RAG vs FT, 20 interview Q&A;

RAG Mastery — Class 1 of 6 | Created by Aman for AmanAI Lab | amanailab.com